

Informe de Pruebas Unitarias

Módulo de Seguridad – RolBusiness

Proyecto: **Mi Cita**

Proyecto: Mi Cita

Tipo: Sistema de agendamiento de citas

Arquitectura: Capas (Controller / Business / Data)

Lenguaje: C# (.NET)

Módulo evaluado: Seguridad

Componente: RolBusiness

Tipo de pruebas: Pruebas Unitarias

Framework de pruebas: xUnit

Mocking: Moq

Fecha: 2025

Introducción

El presente documento describe de manera detallada las pruebas unitarias implementadas para la clase **RolBusiness**, perteneciente al módulo de Seguridad del proyecto **Mi Cita**. Estas pruebas tienen como objetivo validar el correcto funcionamiento de la lógica de negocio de forma aislada, garantizando estabilidad, confiabilidad y mantenibilidad del sistema.

La correcta gestión de roles es crítica dentro del sistema, ya que define los permisos y accesos de los usuarios. Por esta razón, la validación temprana mediante pruebas unitarias resulta fundamental.

Objetivos

Objetivo General

Validar el correcto funcionamiento de la clase **RolBusiness** mediante pruebas unitarias.

Objetivos Específicos

- Verificar la obtención de roles.
 - Validar la creación de nuevos roles.
 - Comprobar la consulta por identificador.
 - Confirmar la eliminación de roles.
 - Garantizar el aislamiento de dependencias.
-

□ Alcance

Las pruebas cubren únicamente la capa **Business**, sin acceso a base de datos ni dependencias externas.

□ Herramientas Utilizadas

- .NET
 - C#
 - xUnit
 - Moq
 - NullLogger
-

□ Casos de Prueba

Obtener todos los roles

```
[Fact]
public async Task GetAll_ShouldReturnListOfRoles()
{
    var roles = new List<Rol>
    {
        new Rol { Id = 1, Name = "Admin" },
        new Rol { Id = 2, Name = "User" }
    };

    _mockRolData.Setup(r => r.GetAll()).ReturnsAsync(roles);

    var result = await _rolBusiness.GetAll();

    Assert.NotNull(result);
    Assert.Equal(2, result.Count());
}
```

Crear un rol

```
[Fact]
public async Task Save_ShouldReturnCreatedRole()
{
    var rolDto = new RolCreatedDto { Name = "NewRole" };
    var rolEntity = new Rol { Id = 10, Name = "NewRole" };

    _mockRolData.Setup(r => r.Save(It.IsAny<Rol>())).ReturnsAsync(rolEntity);

    var result = await _rolBusiness.Save(rolDto);

    Assert.NotNull(result);
    Assert.Equal(10, result.Id);
}
```

Obtener rol por ID

```
[Fact]
public async Task GetById_ShouldReturnRole_WhenExists()
{
    var rolEntity = new Rol { Id = 5, Name = "Manager" };
    _mockRolData.Setup(r => r.GetById(5)).ReturnsAsync(rolEntity);

    var result = await _rolBusiness.GetById(5);

    Assert.NotNull(result);
}
```

Eliminar rol

```
[Fact]
public async Task Delete_ShouldReturnTrue_WhenRoleExists()
{
    _mockRolData.Setup(r => r.Delete(3)).ReturnsAsync(true);

    var result = await _rolBusiness.Delete(3);

    Assert.True(result);
}
```



Resultados

Todos los casos de prueba fueron ejecutados exitosamente, validando el correcto comportamiento de la clase **RolBusiness**.

Conclusiones

Las pruebas unitarias permiten asegurar la calidad del módulo de seguridad del proyecto **Mi Cita**, facilitando el mantenimiento, la detección temprana de errores y la evolución segura del sistema.

Fin del documento